Лабораторна робота №2

Мета роботи: Сформувати практичні навички розробки ноди керування рухом у ROS2, навчитися працювати з повідомленням geometry_msgs/Twist, реалізувати підписку на /cmd_vel, обмеження швидкості та перевірити роботу ноди в симуляції.

Необхідне обладнання

• ПК або ноутбук

• Віртуальна машина з Ubuntu

• Встановлений ROS2 (Humble або узгоджена версія)

• IDE (VS Code / PyCharm / інша)

• Workspace, створений у Лабораторній роботі №1

• Доступ до OneDrive або Git для здачі результатів

1. Підготовка середовища

Роботу виконано в тій самій віртуальній машині та робочому просторі, що й лабораторну роботу №1. ROS2-середовище активовано командами source /opt/ros/humble/setup.bash і source ~/ros2_ws/install/setup.bash. Змінна має значення humble. Команди ros2 node list і ros2 topic list виконуються без помилок. За відсутності запущених нод відображаються лише системні топіки (рис. 1).

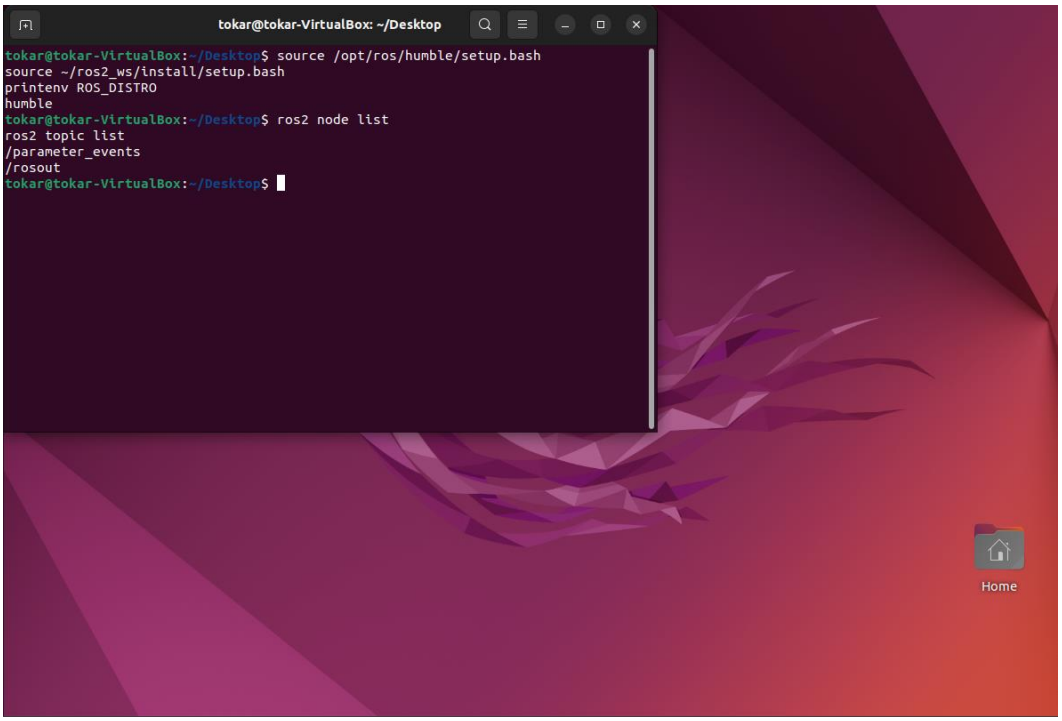


Рисунок 1

Для тестування запущено симулятор turtlesim. Після запуску в системі з'явилися нода /turtlesim і топік керування черепахою /turtle1/cmd_vel.

2. Створення пакету Motion Control

У каталозі ~/ros2_ws/src створено пакет motion_control командою. Пакет має тип збірки ament_python і залежності rclpy та geometry_msgs. Параметр --node-name створив файл ноди motion_node.py і автоматично зареєстрував її в setup.py (рис. 2).

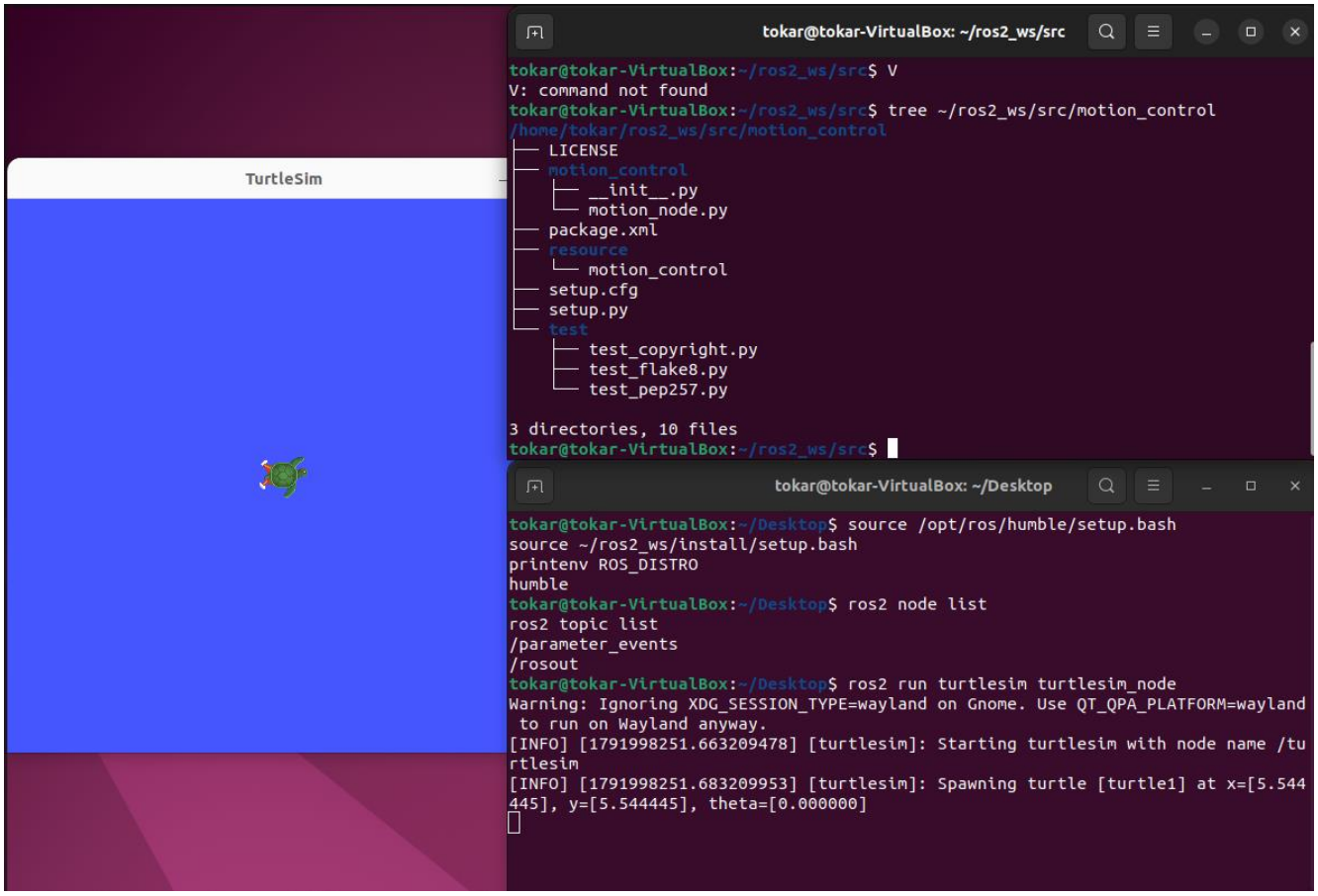


Рисунок 2

Пакет зібрано командою colcon build --packages-select motion_control. Коректність встановлення перевірено командами ros2 pkg list і ros2 pkg executables motion_control, що показали наявність пакету та працюючого файлу motion_node.

3. Реалізація Motion Node

Нода motion_node реалізована класом MotionNode, успадкованим від rclpy.node.Node, і виконує роль проміжного шару безпеки між джерелом команд і роботом. Залежність від geometry_msgs зафіксовано в package.xml, а точку входу зареєстровано в setup.py

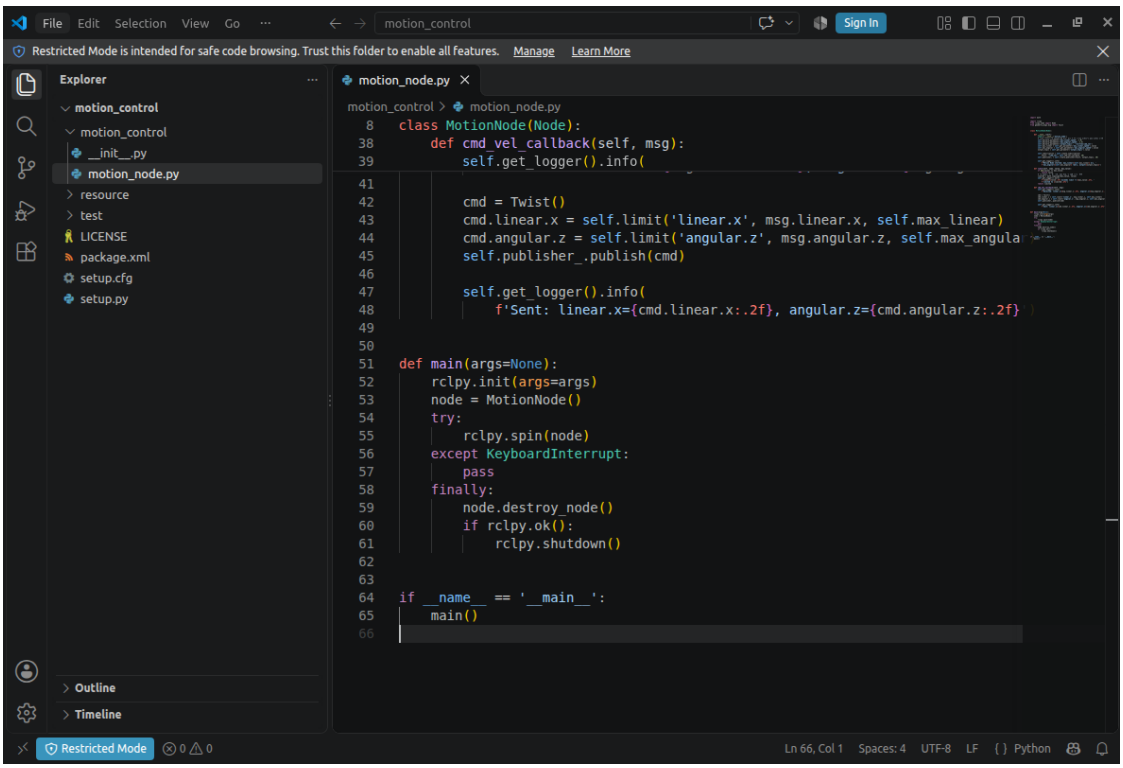


Рисунок 3

4. Тестування ноди

Під час старту нода виводить у журнал встановлені пороги та вихідний топік. Команда ros2 node info /motion_node підтвердила, що нода підписана на /cmd_vel і публікує в /turtle1/cmd_vel (рис. 4).

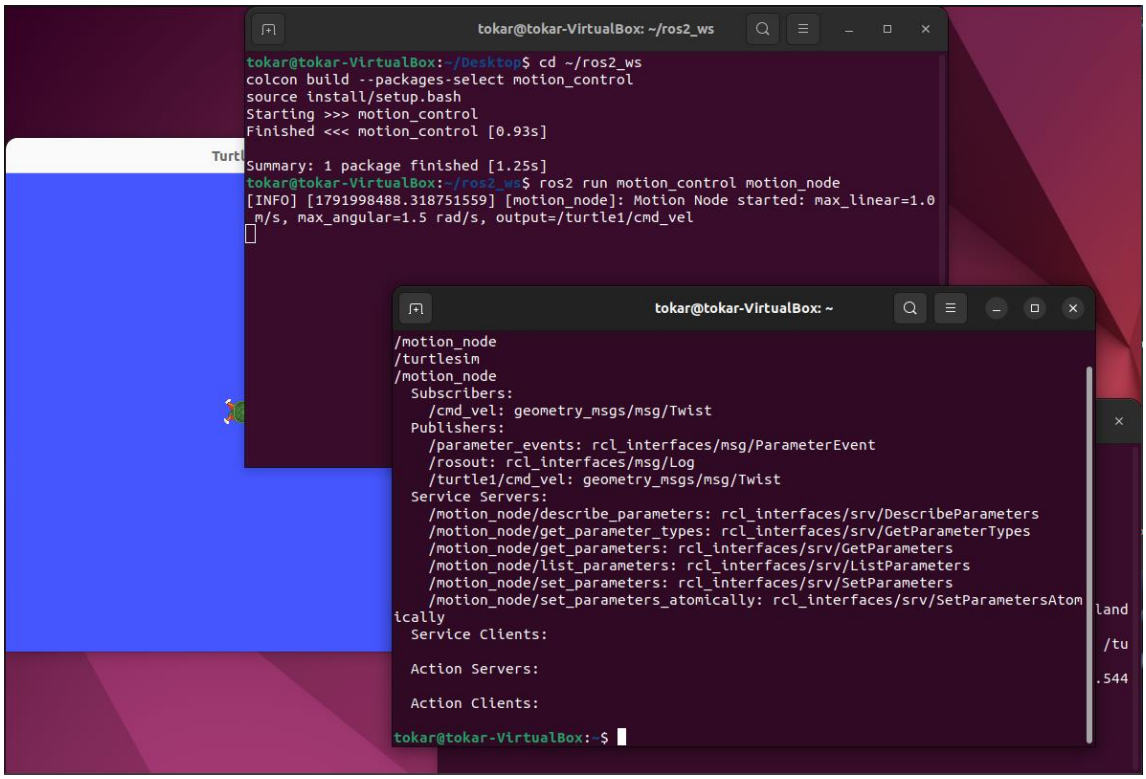


Рисунок 4

Команди в /cmd_vel подавалися двома способами: програмно через ros2 topic pub і вручну через turtle_teleop_key, перенаправлений на топік /cmd_vel (--ros-args -r /turtle1/cmd_vel:=/cmd_vel). Під час подачі команди в межах допустимих значень (linear.x = 0.5, angular.z = 0.5) нода вивела отримані значення на рівні INFO і передала їх у без змін (рис. 5).

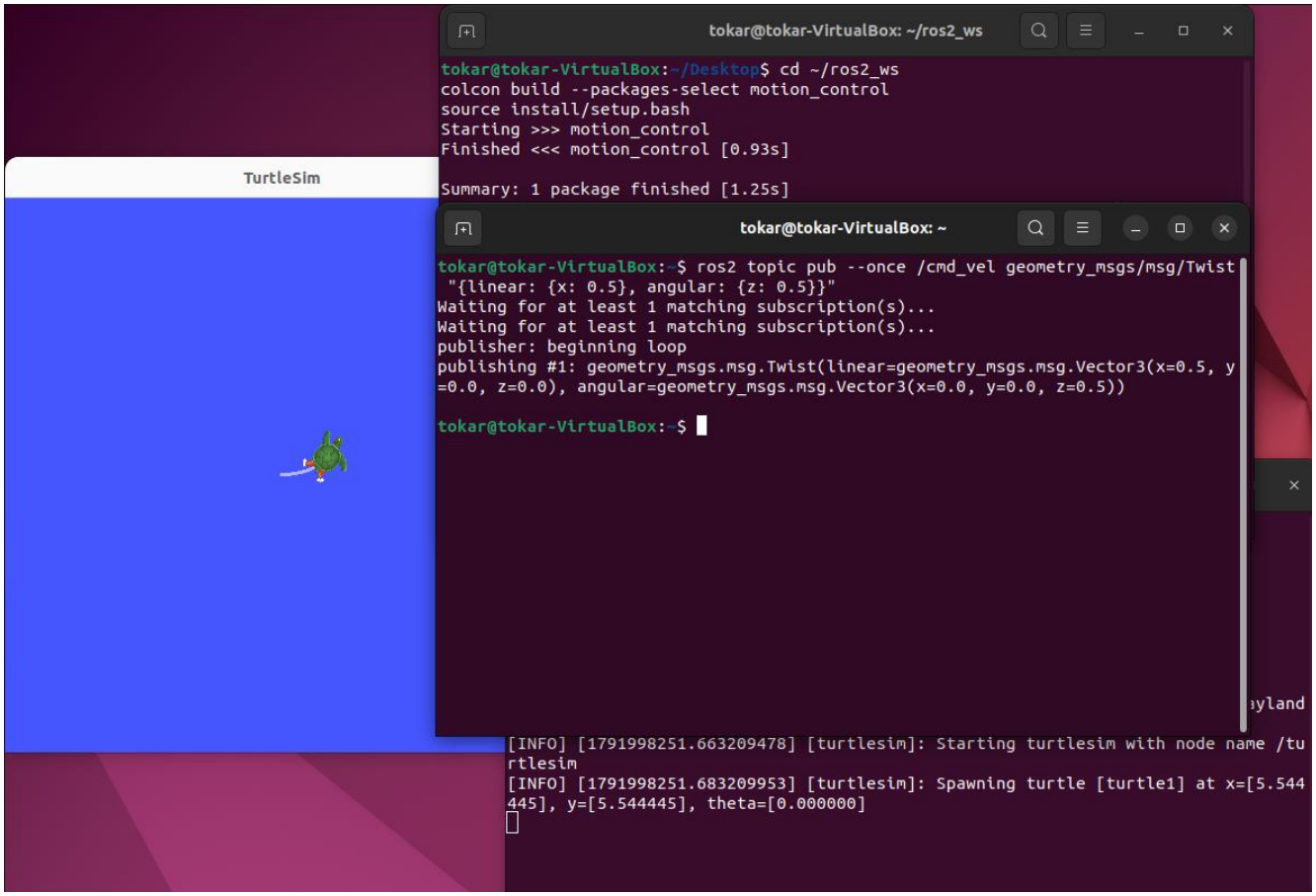


Рисунок 5

Для перевірки обмеження швидкості вказано команди з перевищенням обмежень. В обох випадках нода обмежила значення до зазначеного порогу зі збереженням руху й вивела WARN-повідомлення (рис. 6).

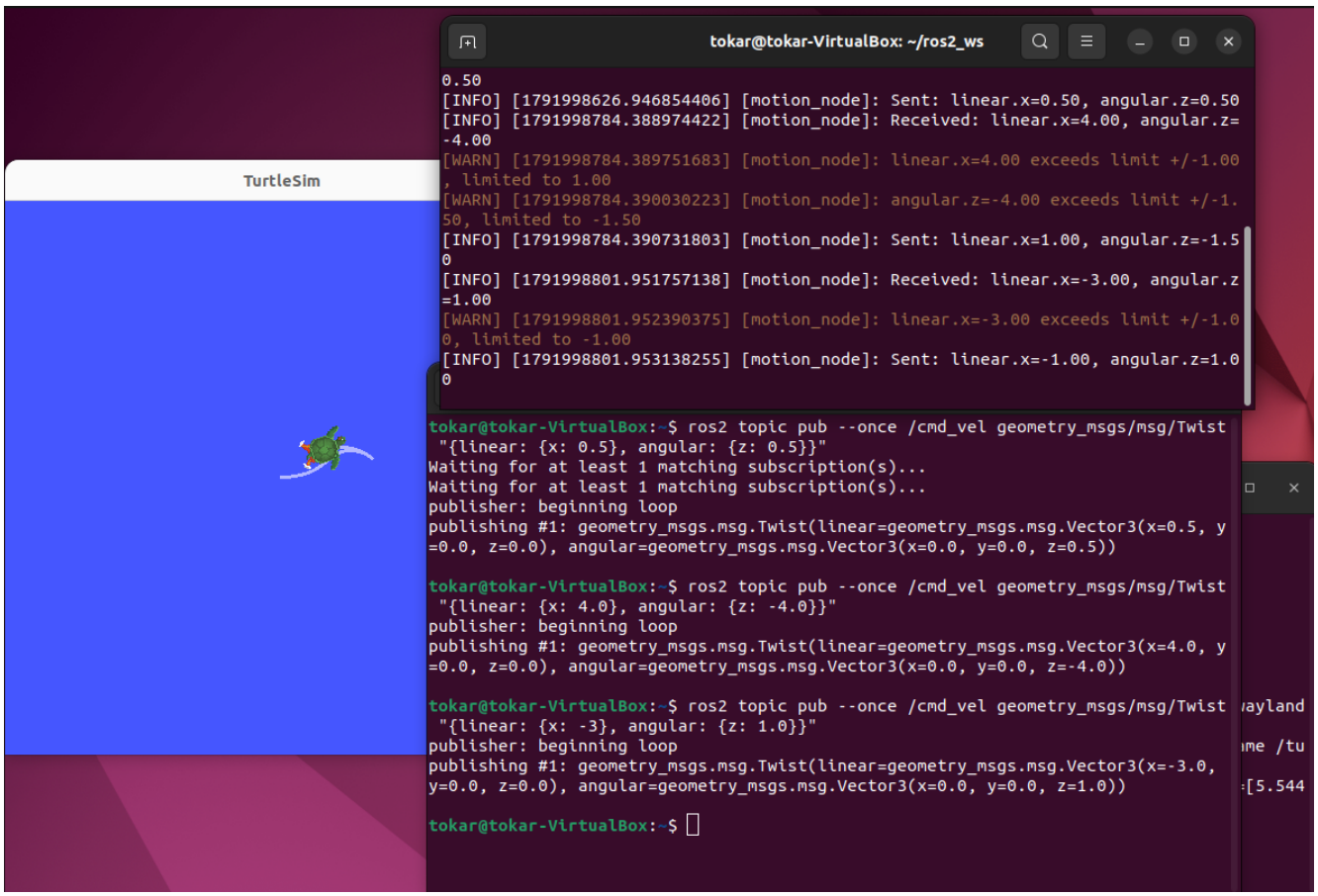


Рисунок 6

Для перевірки через ros2 topic echo одночасно переглядалися вхідний топік ноди і вихідний. При поданні команди linear.x = -3.0, angular.z = 1.0 у вхідному топіку видно вихідні значення, а у вихідному обмежені -1.0 і 1.0 (рис. 7).

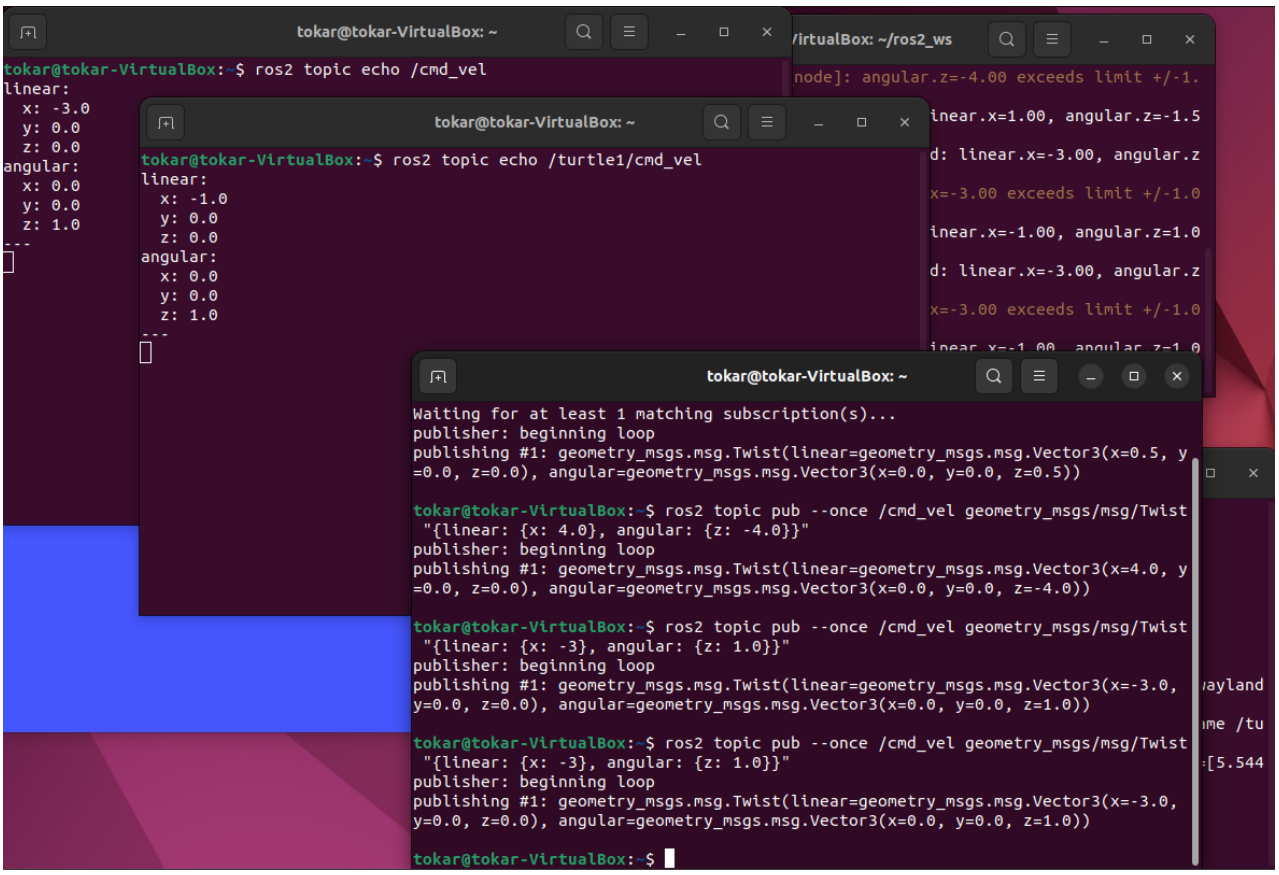
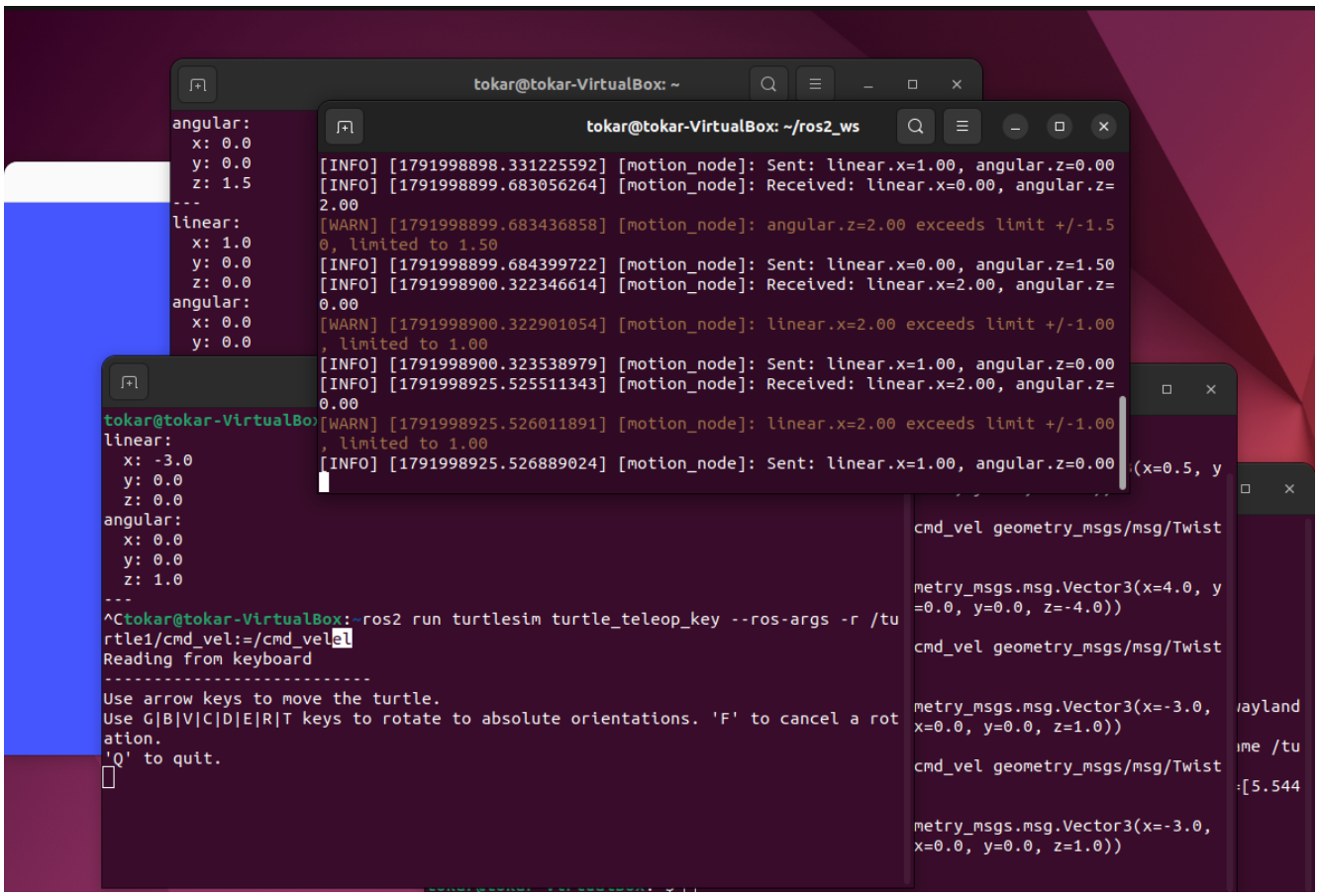


Рисунок 7

Під час ручного керування через teleop кожне натискання клавіші надсилає швидкість 2.0, що перевищує ліміти. Нода обмежувала її до -1.0

(лінійна) та 1.0 (кутова), тому черепаха рухалася помітно повільніше, ніж при прямому керуванні в лабораторній роботі №1 (рис. 8).



Висновок:

Під час виконання лабораторної роботи було створено вузол керування рухом motion_node, що входить до пакета motion_control. Цей вузол приймає повідомлення зі стандартного топіка /cmd_vel, аналізує складові linear.x та angular.z із geometry_msgs/msg/Twist, записує їхні значення до журналу з рівнем INFO, після чого обмежує швидкості відповідно до заданих допустимих меж. Якщо команда виходить за ці межі, вузол повідомляє про це через WARN і надсилає симулятору вже безпечний варіант команди. Перевірку роботи виконано в turtlesim за допомогою команд ros2 topic pub, ros2 topic echo, ros2 node info та засобу візуалізації rqt_graph.

Різниця між ручним і програмним керуванням полягає в тому, хто формує команди. У режимі teleop оператор подає їх безпосередньо через натискання клавіш. Набір таких команд зазвичай обмежений: швидкість фіксована на рівні 2.0, доступні лише рух уперед, назад і поворот. Точність і повторюваність залежать від людської реакції, а в базовому варіанті команди можуть потрапляти до робота без будь-якої перевірки. Перевага такого способу — гнучкість, адже людина може оцінити ситуацію та відреагувати на непередбачувані обставини.

Програмне керування передбачає, що команди створює програма. У цій роботі для цього використовувався ros2 topic pub, а в подальшому це може бути

алгоритм навігації. Значення тут задаються точно, дії є відтворюваними й можуть виконуватися автономно, без участі людини. Проте помилка в програмі здатна сформувати небезпечну команду, тому такий підхід потребує вбудованих механізмів перевірки. Вузол motion_node поєднує обидва підходи. Незалежно від того, звідки надійшла команда — від людини чи від програми — вона проходить програмну перевірку встановлених меж. Завдяки цьому обмеження безпеки зібрано в одному місці, а не дублюються в кожному джерелі команд, і всі випадки перевищення фіксуються в журналі. Крім того, завдяки параметрам, які задають пороги швидкості та вихідний топік, вузол можна перенести на реального робота без змін у коді — достатньо вказати потрібні значення під час запуску.